

AI Agent 的三大核心技术

解构驱动自主智能体的三大基石：MCP，Function Calling与A2A协议



议程：从“大脑”到“行动”的进化之路



1. 新范式：什么是AI Agent?

定义智能体，并阐明其与传统大语言模型的区别。

2. 核心技术一览：连接现实世界的三大催化剂

宏观介绍MCP、Function Calling和A2A协议如何协同工作。



3. 第一催化剂：MCP协议

赋予Agent感知和操作世界的“双手”。



4. 第二催化剂：Function Calling

点燃Agent从思考到行动的“意图火花”。



5. 第三催化剂：A2A协议

构建Agent之间协同工作的“社交网络”。



6. 融会贯通：三大技术如何共同创建一个自主智能体

总结与展望。



新范式：从语言模型（LLM）到人工智能体（AI Agent）

核心定义

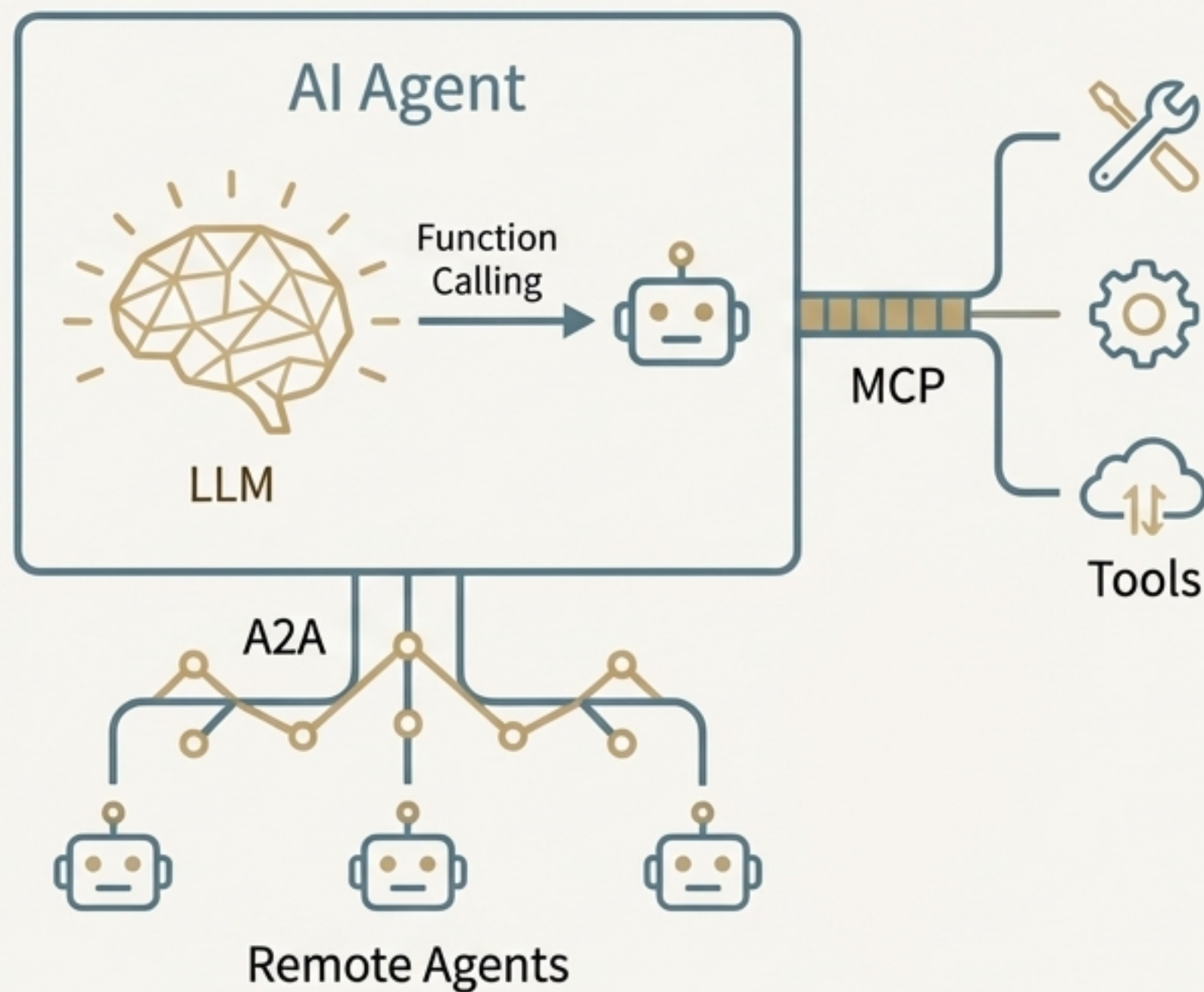
AI Agent是一个基于大语言模型（LLM）的系统，它能够自主地感知环境、进行决策并执行动作。

关系解析

- **LLM是AI Agent的“大脑”**：提供核心的推理、规划和决策能力。
- **AI Agent是LLM的“身体”**：使其能够突破数字世界的束缚，与外部工具和服务进行交互。

关键对比

- **没有Agent的LLM**：一个强大的对话者，但其行动受限于生成的文本。
- **拥有Agent的LLM**：一个能够执行任务的行动者，能将“思考”转化为“行动”。



核心挑战：LLM的“缸中之脑”困境

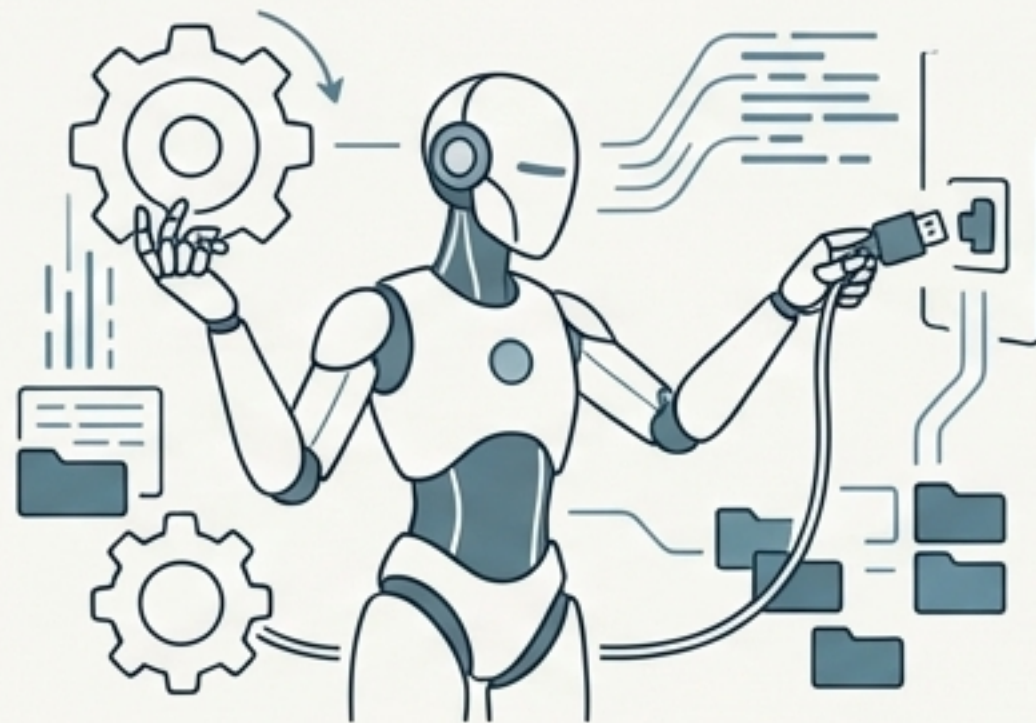
尽管大语言模型（LLM）拥有强大的推理能力，但它们本质上是封闭的系统。它们可以“说”，但不能“做”——无法直接调用API、访问本地文件或其他系统进行交互。

传统SOP/Workflow



基于规则的自动化
流程必须被预先定义
适应性差，行为僵化

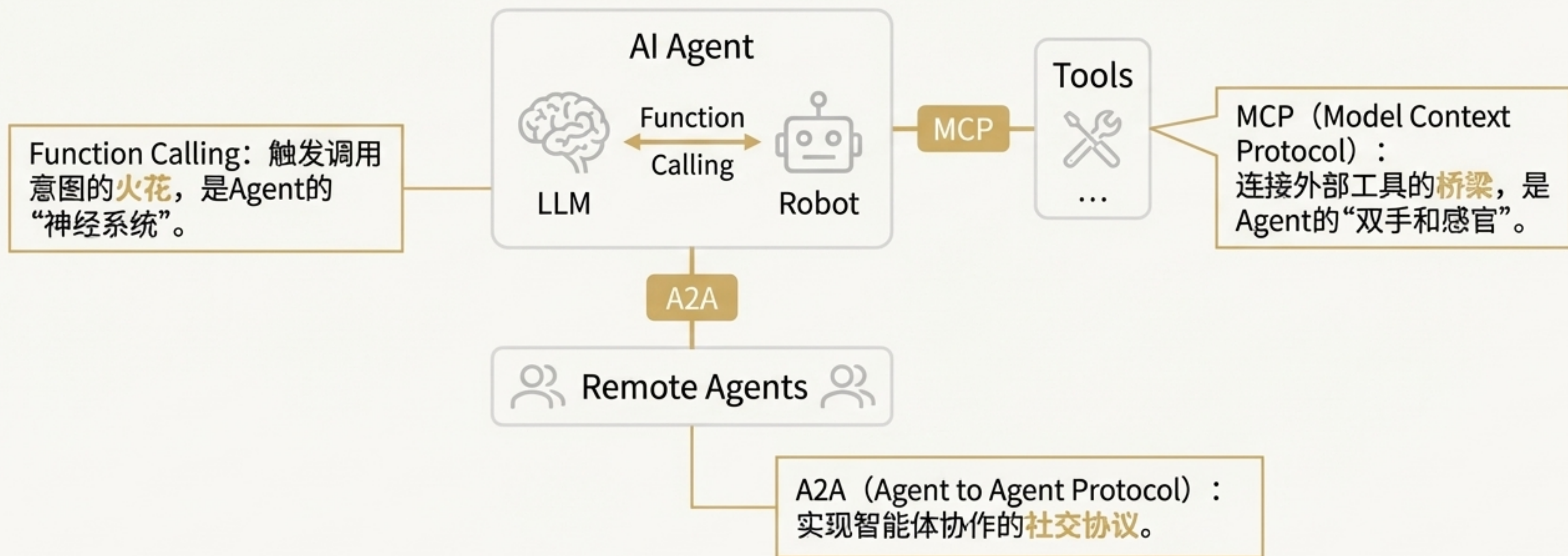
AI Agent



基于目标的自主性
自我驱动、规划、反思
动态适应环境变化，行为灵活

三大核心技术：为“大脑”连接现实世界

为了让LLM能够行动，我们需要一套标准化的协议来连接其“大脑”与外部世界。这套协议由三大核心技术构成。



第一催化剂：MCP - 连接智能体与工具的桥梁

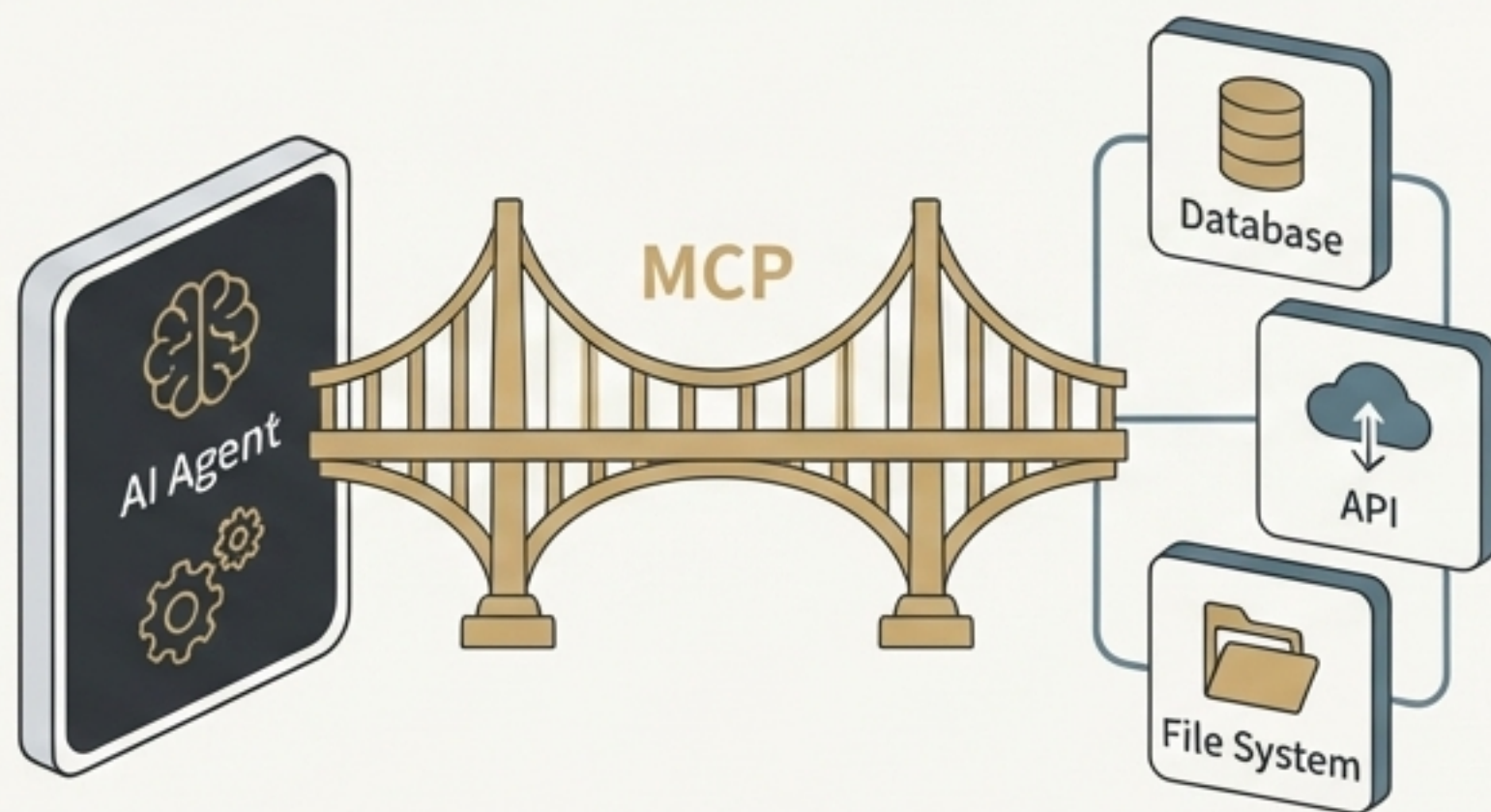
协议定义 (Protocol Definition)

MCP (Model Context Protocol)：一个为AI Agent与外部工具 (Tools) 之间通信而设计的开放、轻量级、可扩展的协议。

核心作用 (Core Function)：它定义了一套标准的“对话”规则，让Agent知道如何发现、理解和调用任何兼容MCP的工具，无论该工具是用什么语言编写或部署在哪里。

重要提示 (Important Note)：

MCP的开发与LLM的选择无关。它是一种通用的工具集成协议，统一了Agent与工具的交互方式。



MCP实战：三步构建并调用一个天气查询工具

第一步：创建MCP工具服务

使用Python和`FastMCP`库，编写一个简单的函数来提供天气信息。

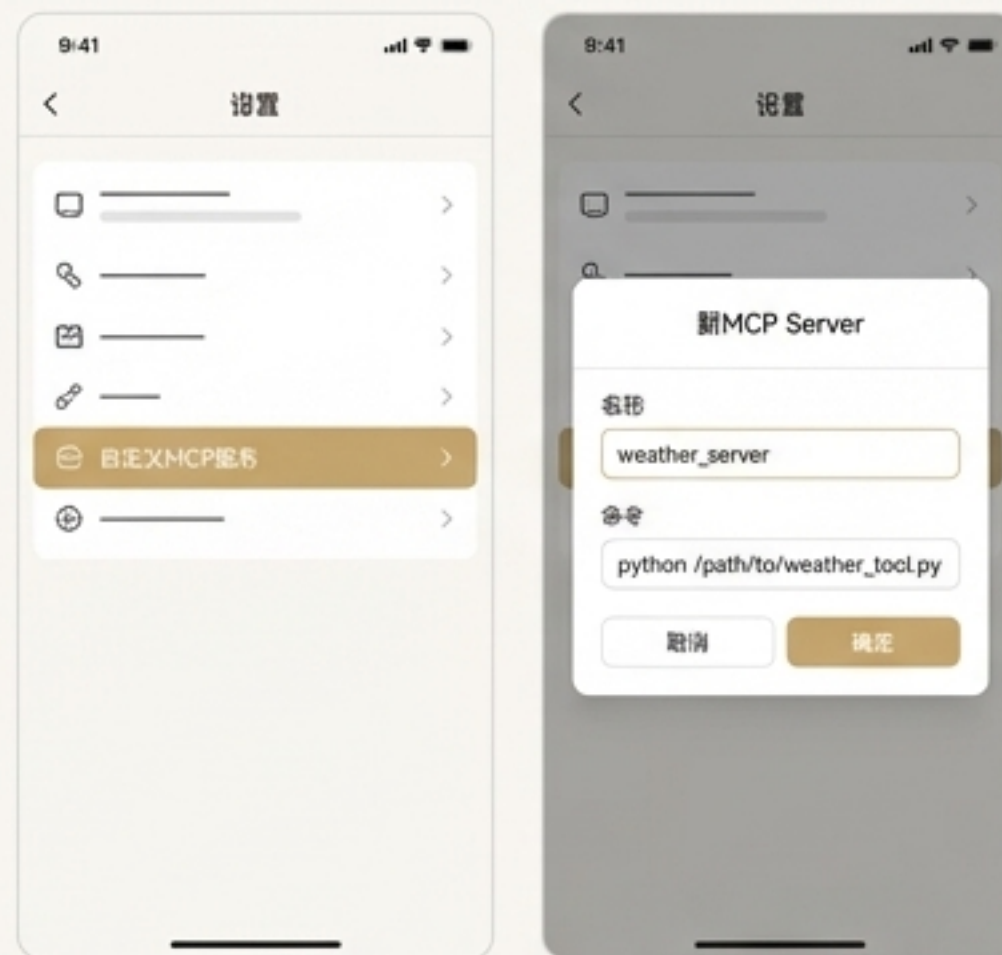
```
# 1. 导入并创建MCP实例
from fastmcp import FastMCP
mcp = FastMCP("获取天气信息")

# 2. 使用@mcp.tool()装饰器定义工具
@mcp.tool("获取天气信息")
def get_forecast(city: str) -> str:
    """获取指定城市的天气信息"""
    return f"{city}的天气明天有大暴雨!" # 模拟

# 3. 运行MCP服务
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

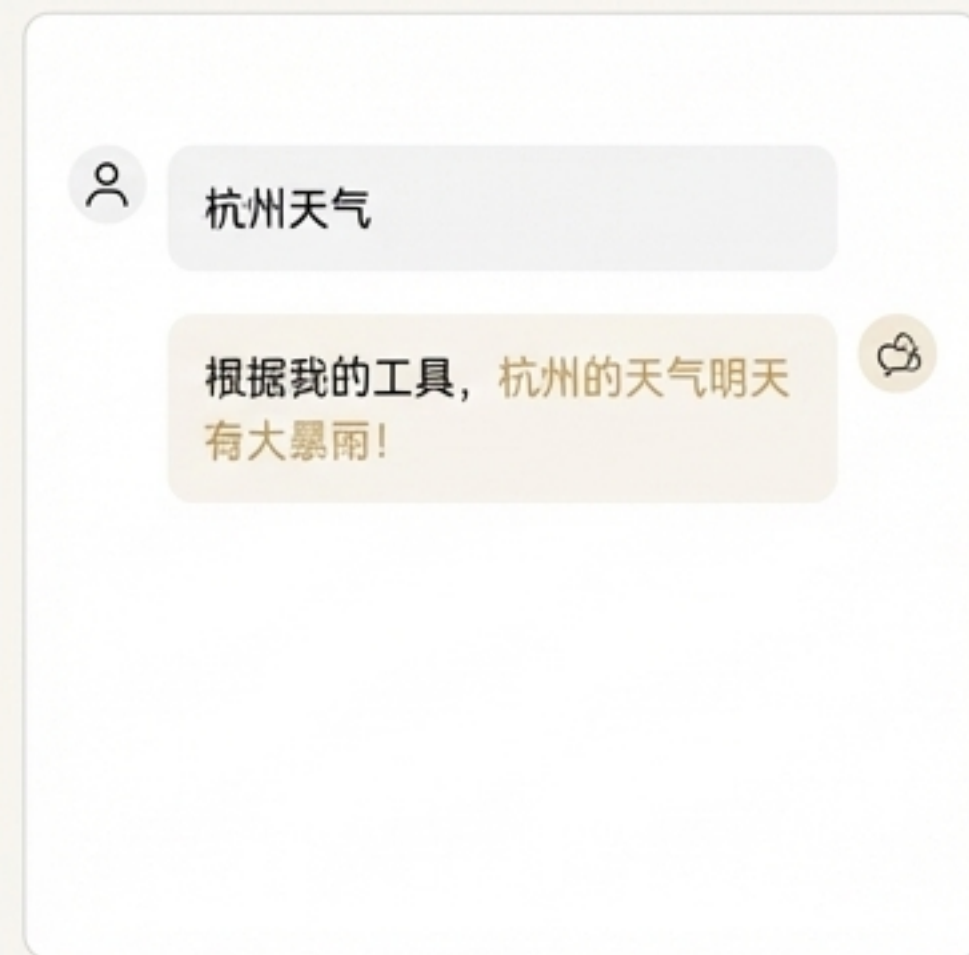
第二步：在客户端配置MCP服务

在支持MCP的客户端（如Chatbox）中，添加我们刚刚创建的本地MCP服务。



第三步：发起调用并验证

在Chatbox中直接提问，观察Agent是否成功调用了我们的工具。



深入解析：MCP的JSON-RPC通信流程

MCP通信基于JSON-RPC协议，整个交互过程分为三个清晰的阶段。

1. 初始化阶段（握手）

目的：客户端与服务器建立连接，交换协议版本和能力信息。

交互：Client 发送 initialize 请求，Server 回复 initialize 结果。

```
Client ->: {"method": "initialize", "params": {...}}
Server <-: {"result": {"protocolVersion": "...", "capabilities": ...}}
```

2. 工具注册阶段

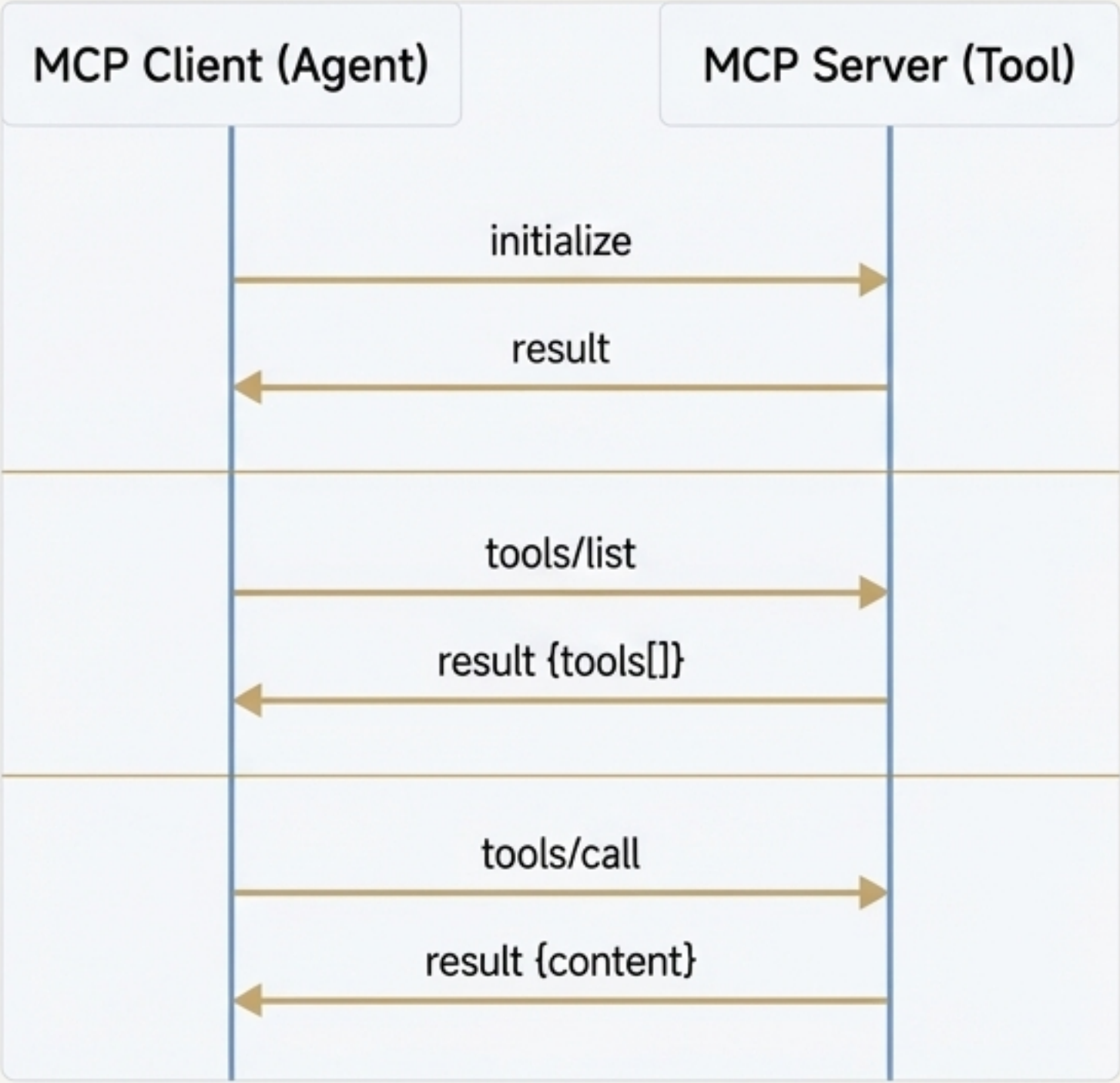
目的：服务器向客户端通告其可用的工具列表及其详细信息（名称、描述等）。

交互：Client 发送 tools/list 请求，Server 回复包含工具定义的 result。

3. 工具调用阶段

目的：客户端请求执行某个特定的工具，并传递参数。

交互：Client 发送 tools/call 请求（包含工具名和参数），Server 执行工具并返回结果。



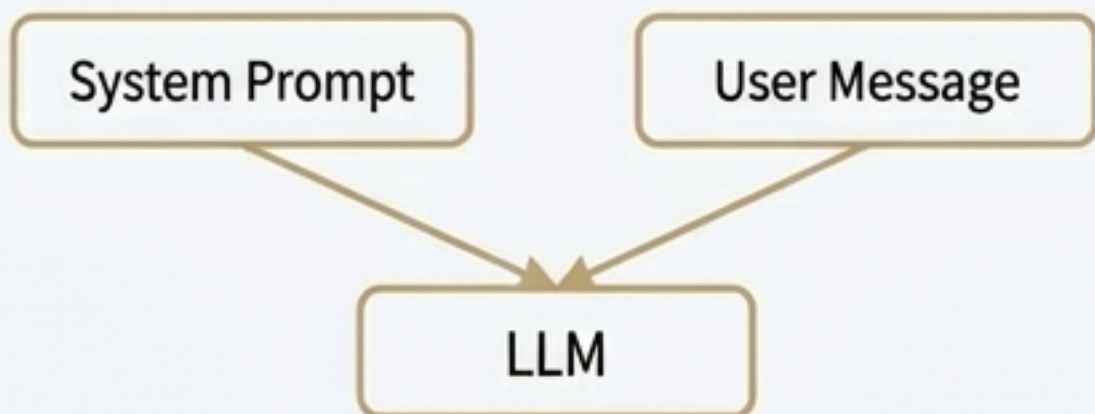
第二催化剂：Function Calling - 将思考转化为意图

MCP提供了连接工具的“管道”，但LLM（大脑）如何知道**何时、为何以及如何**去使用这些工具呢？

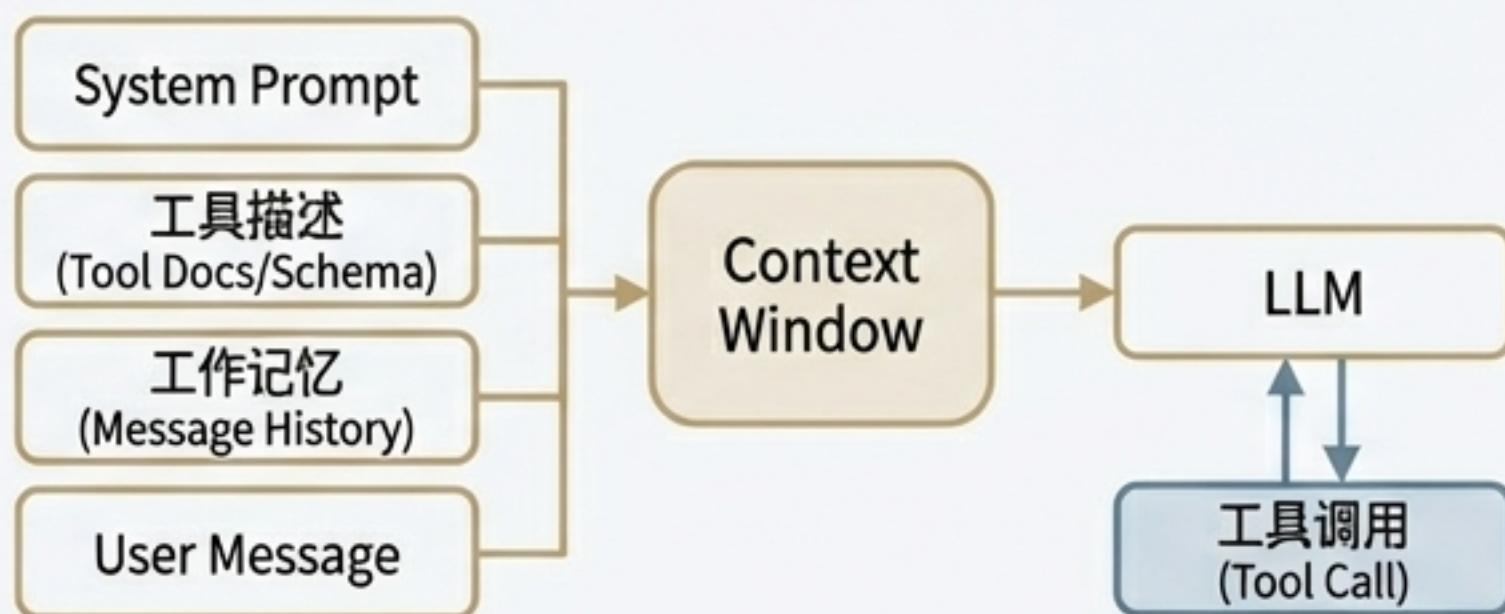
Function Calling。它不是一个协议，而是一种**技术**，通过精巧的提示工程（Prompt Engineering）和结构化的工具定义，让LLM能够理解并生成调用工具的意图。

从Prompt Engineering到Context Engineering

Prompt Engineering（单轮查询）



Context Engineering (Agent)



Function Calling的实现：Prompt与Schema双管齐下

为LLM编写“说明书”

通过系统提示（System Prompt），赋予LLM角色，并教会它如何使用工具。

身份声明

You are Cline, a highly skilled software engineer...

能力声明（如何使用工具）

TOOL USE

You have access to a set of tools...

Tool use is formatted using XML style tags.

<tool_code>

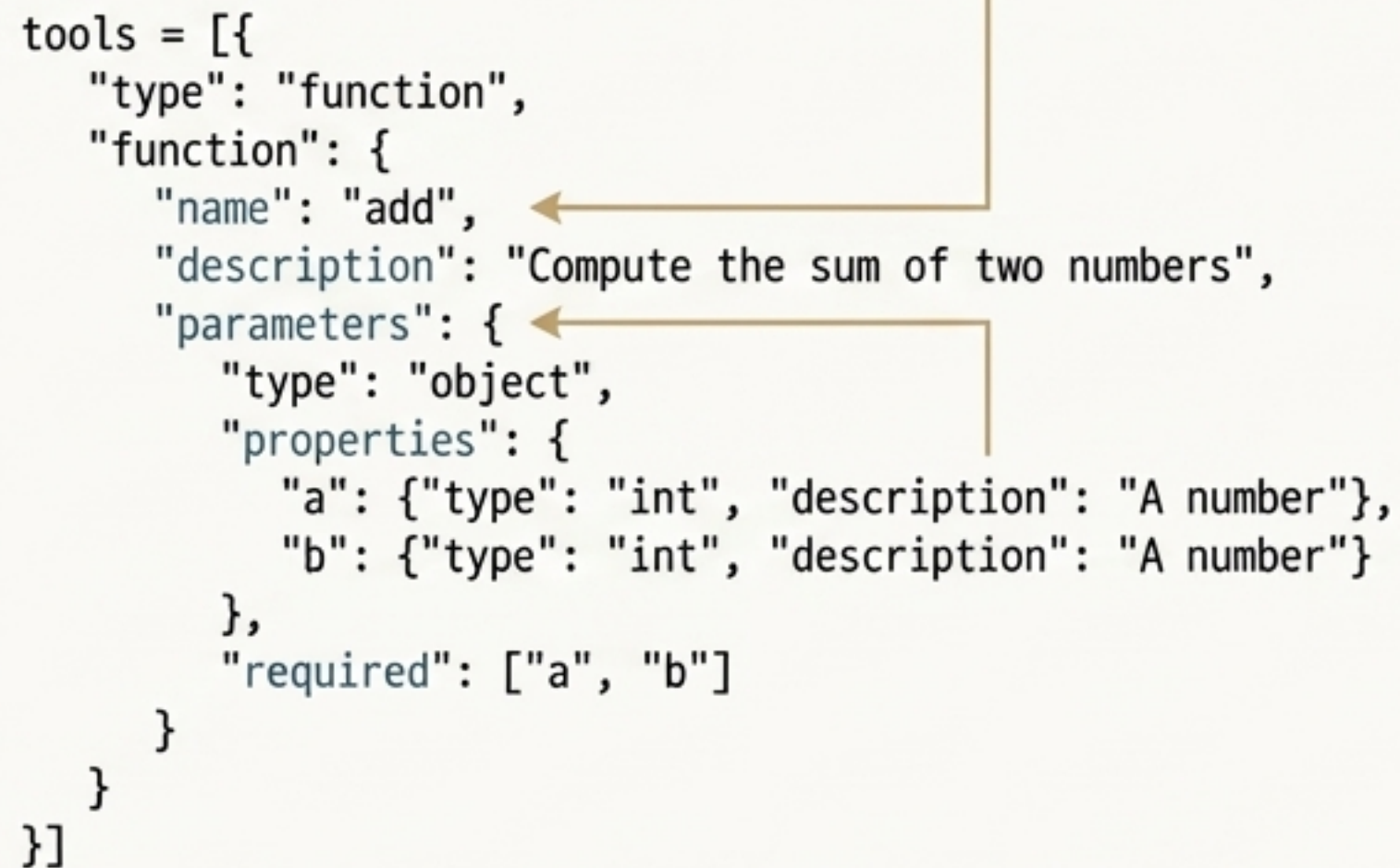
The tool code to execute

</tool_code>

为工具定义“身份证”

将工具以结构化的JSON Schema格式提供给LLM，使其能够准确理解工具的功能和参数。

```
tools = [{  
  "type": "function",  
  "function": {  
    "name": "add",  
    "description": "Compute the sum of two numbers",  
    "parameters": {  
      "type": "object",  
      "properties": {  
        "a": {"type": "int", "description": "A number"},  
        "b": {"type": "int", "description": "A number"}  
      },  
      "required": ["a", "b"]  
    }  
  }  
}]
```



第三催化剂：A2A协议 - 从“个体”到“协作网络”

待解决的问题

- 上下文长度限制：单个Agent的记忆和处理能力有限，无法处理极其复杂的任务。
- 任务专业化：许多复杂任务需要不同领域的专业知识，最好由多个专门的Agent协作完成。

解决方案

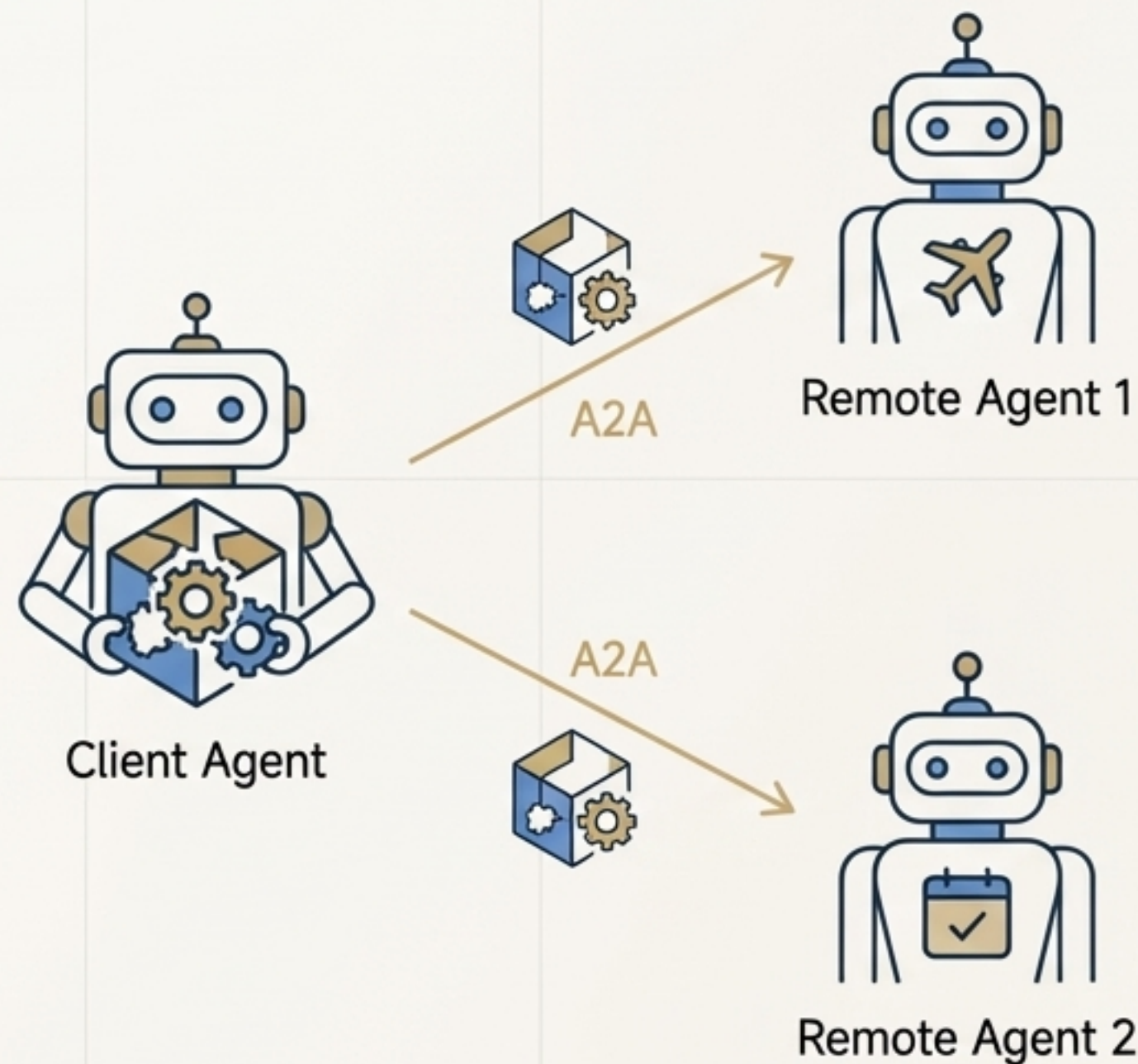
A2A (Agent to Agent) 协议。一个用于AI Agent之间发现、通信和任务协作的协议。

核心理念

- Agent Card：Agent的“名片”，用于自我介绍和能力发现。
- 任务委派 (Task Delegation)：一个Client Agent可以将一个复杂的任务分解，并委派给多个专精的Remote Agent。

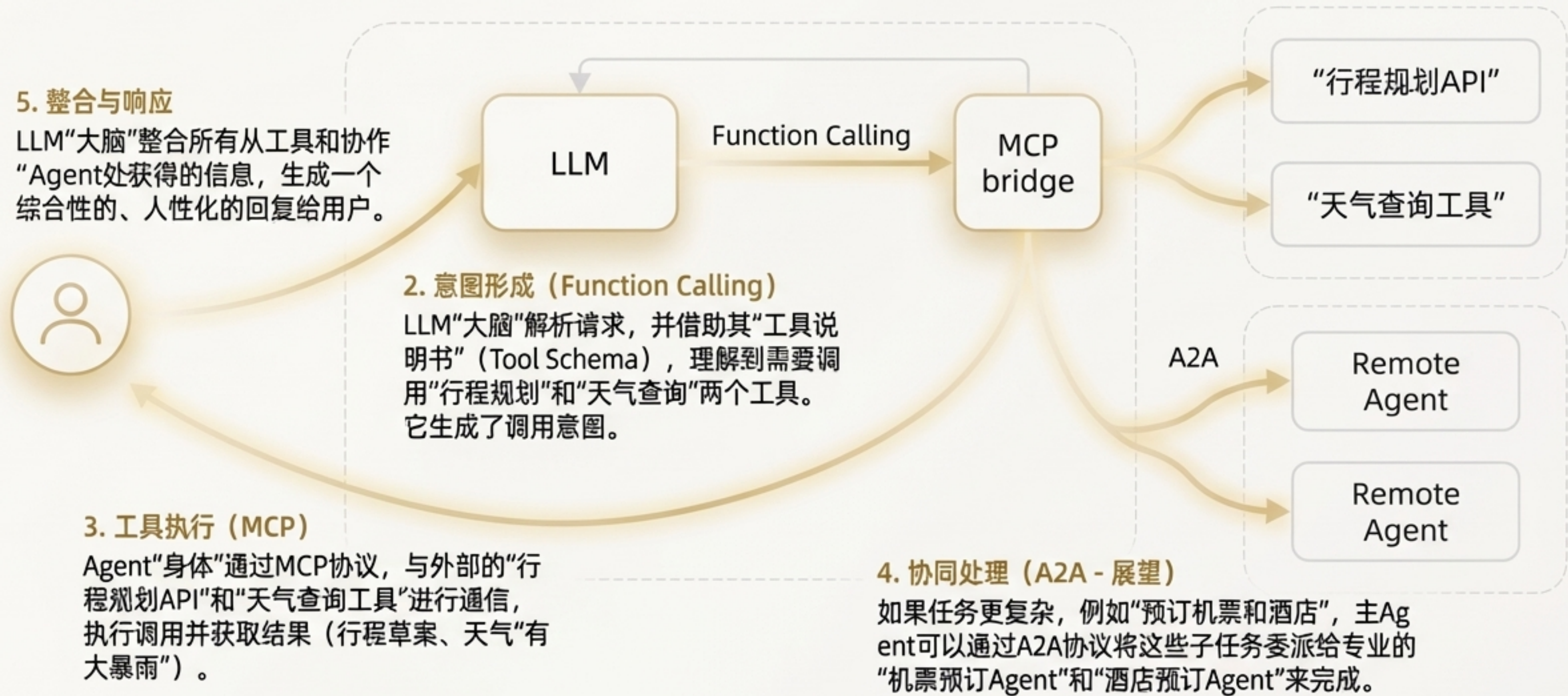
当前状态

A2A协议仍处于早期发展阶段，尚未形成统一的成熟标准，但代表了Agent技术未来的重要方向。

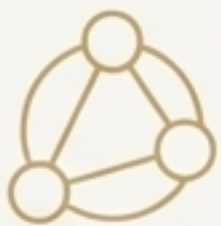


融会贯通：一个自主智能体的诞生

一个完整请求的生命周期

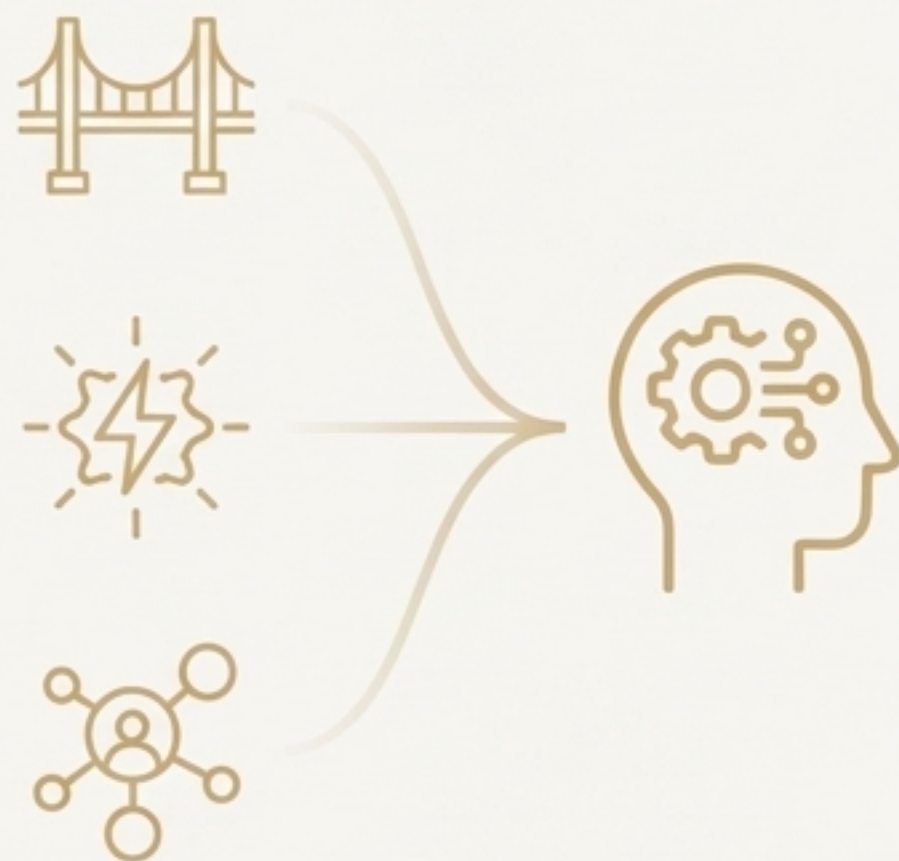


总结：三大核心技术的关系与分工

技术 (Technology)	核心作用 (Core Function)	交互关系 (Interaction Mapping)	隐喻 (Metaphor)
MCP 	抽象化和标准化工具的接入与调用，提供物理连接。	AI Agent <---> AI Tools	双手与感官 (Hands & Senses)
Function Calling 	通过Prompt和Schema指导大模型理解并决定如何使用工具。	AI Model <---> Tools	神经系统 (Nervous System)
A2A 	实现Agent间的任务分解与协作，突破个体能力上限。	AI Agent <---> AI Agent	社交协议 (Social Protocol)

核心价值：从“对话”到“行动”的飞跃

- **MCP**：通过标准化的工具集成，赋予了Agent执行物理世界或数字世界任务的实践能力。
- **Function Calling**：通过巧妙的上下文工程，弥合了LLM抽象推理能力与具体工具执行能力之间的鸿沟。
- **A2A**：通过Agent间的协作委托，解决了单一Agent上下文长度和专业能力的限制，为解决超大规模复杂问题提供了可能。



这三大技术的融合，正是将大语言模型从一个令人惊艳的“对话者”转变为能够自主完成复杂任务的“行动者”的关键。它们共同构筑了通往通用人工智能（AGI）的重要阶梯。